

Section 33: PostgreSQL

247. Introduction to Postgres

- PostgreSQL is an open-source, object-relational database management system (ORDBMS) known for its robustness, extensibility, and standards compliance. It supports SQL for querying and managing data and provides powerful features such as:
 - ACID Compliance: Ensures reliable transactions with strong data integrity and consistency.
 - Extensibility: Allows users to define custom data types, operators, and functions, making it highly customizable.
 - Advanced Features: Supports advanced data types (JSON, XML, arrays), full-text search, and geospatial data with PostGIS.
 - Concurrency Control: Utilizes Multi-Version Concurrency Control (MVCC) to handle high levels of concurrent transactions without locking.
 - Cross-Platform: Available on various operating systems, including Windows, macOS, and Linux.
 - Community and Support: Backed by a strong community and numerous extensions, it offers extensive support and documentation.
- PostgreSQL is commonly used for web applications, data warehousing, and complex applications that require advanced data handling capabilities.
- Why PostgreSQL?
 - Free and open source
 - Widely used in industry
 - Community support
 - Career opportunities
- How to use PostgreSQL?

```
import Client from "pg";
const DB = new Client({
  user: "username",
  host: "localhost",
  database: "myDatabase",
  password: "password",
  port: 5432
});
DB.connect();

DB.query("SELECT * FROM users", (err, res) => {
  if (err) console.error("Error executing query", err.stack);
  else console.log("User data: ", res.rows);
  DB.end();
});
```

- Package name: `pg` (node-postgres).
- Pre-requisites:

- Postgres Server: Stands up a Postgres server on our local computer.
- pgAdmin: UI to tap into our Postgres server.

249. Understand how to use keys, Postgres types and keywords

- Postgres Data Types: <https://www.postgresql.org/docs/current/datatype.html>
- CSV – Comma Separated Values.

253. Query data using SELECT, WHERE and LIKE

- Various select methods:

```
1 create table world_food (
2     country text,
3     rice float,
4     wheat float
5 );
6 select * from world_food;
7 select country from world_food;
8 select country, wheat from world_food;
9 select rice from world_food where country = 'United States';
10 select country from world_food where wheat > 20;
11 select * from world_food where country like 'U' || '%';
12 select * from world_food where country like '%' || 'a';
```

254. UNIQUE and NOT NULL | Travel Tracker Part 1

- NOT NULL – Missing or empty values are not allowed. Attempts to ass NULL will result in an error.
- UNIQUE – Value cannot be repeated in the table. No other value stored can be the same (avoids duplicates).

255. INSERT and add Data | Travel Tracker Part 2

```
await DB.query(
    "INSERT INTO visited_countries (country_code) VALUES ($1)",
    [countryCode]
);
```

257. One to One Relationships and Inner Joins

```
1 create table student (
2     id serial primary key,
3     first_name text,
4     last_name text
5 );
6 create table contact_detail (
7     id integer references student(id) unique,
8     telephone text,
9     address text
10 );
```

- Example of one-to-one relationship.

```

18
19 ✓ SELECT *
20 FROM student
21 JOIN contact_detail|
22 ON student.id = contact_detail.id

```

- Inner join

258. One to Many Relationships

```

1 ✓ CREATE TABLE student (
2     id SERIAL PRIMARY KEY,
3     first_name TEXT,
4     last_name TEXT
5 );
6 ✓ CREATE TABLE homework_submission (
7     id SERIAL PRIMARY KEY,
8     mark INTEGER,
9     student_id INTEGER REFERENCES student(id)
10 );|

```

- One to many example.

```

SELECT *
FROM student
JOIN homework_submission
ON student.id = homework_submission.student_id

```

- One to many join.

259. Many to Many Relationships and Aliases

```

1 ✓ CREATE TABLE student (
2     id SERIAL PRIMARY KEY,
3     first_name TEXT,
4     last_name TEXT
5 );
6 ✓ CREATE TABLE class (
7     id SERIAL PRIMARY KEY,
8     title VARCHAR(45)
9 );
10
11 ✓ CREATE TABLE enrollment (
12     student_id INTEGER REFERENCES student(id),
13     class_id INTEGER REFERENCES class(id),
14     PRIMARY KEY (student_id, class_id)
15 );|

```

- Many to many example.

```
50
51 SELECT *
52 FROM student
53 JOIN homework_submission
54 ON student.id = homework_submission.student_id
55
56 SELECT *
57 FROM enrollment
58 JOIN student ON student.id = enrollment.student_id
59 JOIN class ON class.id = enrollment.class_id;
```

- Many to many join.